Smart Lender - Loan Eligibility Prediction

Machine learning web application for loan approval prediction.

Generated on 2026-07-14 10:36:00

Smart Lender predicts loan eligibility from applicant data using preprocessed features and a trained model. The report includes project architecture, setup details, dataset contents, and complete source files.

Contents

Project Overview

Smart Lender - Loan Eligibility Prediction System

Smart Lender is a machine learning-powered loan eligibility prediction app built with Python and Flask. It evaluates applicant information using multiple classifiers and deploys the best model for real-time prediction.

Features - Preprocess loan applicant data - Train Decision Tree, Random Forest, KNN, and XGBoost models - Select and save the best-performing model - Provide a Flask web interface for real-time loan eligibility prediction

Setup 1. Create a virtual environment: ```bash python -m venv venv source venv/Scripts/activate # Windows # or source venv/bin/activate # Linux/macOS ``` 2. Install dependencies: ```bash pip install -r requirements.txt ``` 3. Train the model: ```bash python train_model.py ``` 4. Run the Flask app: ```bash python app.py ``` 5. Open `http://127.0.0.1:5000` in your browser.

Data Place a loan dataset at `data/loan_data.csv` with the following columns: - Gender - Married - Education - Self_Employed - ApplicantIncome - CoapplicantIncome - LoanAmount - Loan_Amount_Term - Credit_History - Property_Area - Loan_Status

If no dataset is present, the app will use a built-in sample dataset for training.

Notes - The project saves the trained model to `models/best_loan_model.pkl`. - You can replace the sample data with your own loan eligibility dataset and re-run training.

Setup and Requirements

Install required Python packages, train the model, and launch the Flask web application.

Recommended commands:

```
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt
python train_model.py
python app.py
```

Data columns used in the loan dataset:

Gender

Married

Education

Self_Employed

ApplicantIncome

CoapplicantIncome

LoanAmount

Loan_Amount_Term

Credit_History

Property_Area

Loan_Status

Included Project Files

README.md
requirements.txt
train_model.py
app.py
.gitignore
data\loan_data.csv
templates\index.html
templates\result.html

File: README.md

Full file contents:

````
# Smart Lender - Loan Eligibility Prediction System

Smart Lender is a machine learning-powered loan eligibility prediction app built with Python and Flask. It evaluate

## Features
- Preprocess loan applicant data
- Train Decision Tree, Random Forest, KNN, and XGBoost models
- Select and save the best-performing model
- Provide a Flask web interface for real-time loan eligibility prediction

## Setup
1. Create a virtual environment:
   ```bash
   python -m venv venv
   source venv/Scripts/activate   # Windows
   # or source venv/bin/activate   # Linux/macOS
   ```
2. Install dependencies:
   ```bash
   pip install -r requirements.txt
   ```
3. Train the model:
   ```bash
   python train_model.py
   ```
4. Run the Flask app:
   ```bash
   python app.py
   ```
5. Open `http://127.0.0.1:5000` in your browser.

## Data
Place a loan dataset at `data/loan_data.csv` with the following columns:
- Gender
- Married
- Education
- Self_Employed
- ApplicantIncome
- CoapplicantIncome
- LoanAmount
- Loan_Amount_Term
- Credit_History
- Property_Area
- Loan_Status

If no dataset is present, the app will use a built-in sample dataset for training.

## Notes
- The project saves the trained model to `models/best_loan_model.pkl`.
- You can replace the sample data with your own loan eligibility dataset and re-run training.
````

File: requirements.txt

Full file contents:

```
Flask==2.3.3
pandas==2.2.3
numpy==1.26.5
scikit-learn==1.3.3
xgboost==1.8.2
joblib==1.3.2
matplotlib==3.9.4
seaborn==0.13.2
```

File: train_model.py

Full file contents:

```
import os
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.neighbors import KNeighborsClassifier
import xgboost as xgb
from sklearn.metrics import accuracy_score
import joblib

DATA_PATH = os.path.join("data", "loan_data.csv")
MODEL_DIR = "models"
MODEL_PATH = os.path.join(MODEL_DIR, "best_loan_model.pkl")

CATEGORICAL_COLUMNS = [
    "Gender",
    "Married",
    "Education",
    "Self_Employed",
    "Credit_History",
    "Property_Area",
]
NUMERIC_COLUMNS = [
    "ApplicantIncome",
    "CoapplicantIncome",
    "LoanAmount",
    "Loan_Amount_Term",
]
TARGET_COLUMN = "Loan_Status"

SAMPLE_DATA = [
    {
        "Gender": "Male",
        "Married": "Yes",
        "Education": "Graduate",
        "Self_Employed": "No",
        "ApplicantIncome": 5849,
        "CoapplicantIncome": 0.0,
        "LoanAmount": 128.0,
        "Loan_Amount_Term": 360.0,
        "Credit_History": 1,
        "Property_Area": "Urban",
        "Loan_Status": "Y",
    },
    {
        "Gender": "Male",
        "Married": "Yes",
        "Education": "Graduate",
        "Self_Employed": "No",
        "ApplicantIncome": 4583,
        "CoapplicantIncome": 1508.0,
        "LoanAmount": 128.0,
        "Loan_Amount_Term": 360.0,
        "Credit_History": 1,
        "Property_Area": "Rural",
        "Loan_Status": "N",
    },
```

```
{
    "Gender": "Male",
    "Married": "Yes",
    "Education": "Not Graduate",
    "Self_Employed": "Yes",
    "ApplicantIncome": 3000,
    "CoapplicantIncome": 0.0,
    "LoanAmount": 66.0,
    "Loan_Amount_Term": 360.0,
    "Credit_History": 1,
    "Property_Area": "Urban",
    "Loan_Status": "Y",
},
{
    "Gender": "Female",
    "Married": "No",
    "Education": "Graduate",
    "Self_Employed": "No",
    "ApplicantIncome": 6000,
    "CoapplicantIncome": 0.0,
    "LoanAmount": 141.0,
    "Loan_Amount_Term": 360.0,
    "Credit_History": 1,
    "Property_Area": "Urban",
    "Loan_Status": "Y",
},
{
    "Gender": "Male",
    "Married": "No",
    "Education": "Graduate",
    "Self_Employed": "Yes",
    "ApplicantIncome": 5417,
    "CoapplicantIncome": 4196.0,
    "LoanAmount": 267.0,
    "Loan_Amount_Term": 360.0,
    "Credit_History": 1,
    "Property_Area": "Urban",
    "Loan_Status": "Y",
},
{
    "Gender": "Male",
    "Married": "Yes",
    "Education": "Not Graduate",
    "Self_Employed": "No",
    "ApplicantIncome": 2333,
    "CoapplicantIncome": 1516.0,
    "LoanAmount": 95.0,
    "Loan_Amount_Term": 360.0,
    "Credit_History": 1,
    "Property_Area": "Rural",
    "Loan_Status": "N",
},
{
    "Gender": "Female",
    "Married": "Yes",
    "Education": "Graduate",
    "Self_Employed": "No",
    "ApplicantIncome": 3036,
    "CoapplicantIncome": 2504.0,
    "LoanAmount": 158.0,
    "Loan_Amount_Term": 360.0,
    "Credit_History": 1,
    "Property_Area": "Urban",
    "Loan_Status": "Y",
},
{
    "Gender": "Male",
```

```
        "Married": "No",
        "Education": "Graduate",
        "Self_Employed": "No",
        "ApplicantIncome": 4006,
        "CoapplicantIncome": 1526.0,
        "LoanAmount": 168.0,
        "Loan_Amount_Term": 360.0,
        "Credit_History": 1,
        "Property_Area": "Urban",
        "Loan_Status": "N",
    },
    {
        "Gender": "Male",
        "Married": "No",
        "Education": "Not Graduate",
        "Self_Employed": "No",
        "ApplicantIncome": 12841,
        "CoapplicantIncome": 10968.0,
        "LoanAmount": 350.0,
        "Loan_Amount_Term": 360.0,
        "Credit_History": 1,
        "Property_Area": "Urban",
        "Loan_Status": "Y",
    },
    {
        "Gender": "Female",
        "Married": "Yes",
        "Education": "Not Graduate",
        "Self_Employed": "No",
        "ApplicantIncome": 3200,
        "CoapplicantIncome": 0.0,
        "LoanAmount": 200.0,
        "Loan_Amount_Term": 360.0,
        "Credit_History": 0,
        "Property_Area": "Semiurban",
        "Loan_Status": "N",
    },
]


def load_data():
    if os.path.exists(DATA_PATH):
        print(f"Loading dataset from {DATA_PATH}")
        return pd.read_csv(DATA_PATH)

    print("Dataset not found. Using sample loan data for training.")
    return pd.DataFrame(SAMPLE_DATA)


def preprocess_features(df):
    df = df.copy()
    if TARGET_COLUMN not in df.columns:
        raise ValueError(f"Dataset must include the target column '{TARGET_COLUMN}'.")

    df[TARGET_COLUMN] = df[TARGET_COLUMN].map({"Y": 1, "N": 0, 1: 1, 0: 0})
    df = df.dropna(subset=CATEGORICAL_COLUMNS + NUMERIC_COLUMNS + [TARGET_COLUMN])

    X = df[CATEGORICAL_COLUMNS + NUMERIC_COLUMNS]
    y = df[TARGET_COLUMN].astype(int)
    return X, y


def build_pipeline(classifier):
    numeric_transformer = Pipeline(
        steps=[
            ("imputer", SimpleImputer(strategy="median")),
            ("scaler", StandardScaler()),
```

```
        ]
    )

    categorical_transformer = Pipeline(
        steps=[
            ("imputer", SimpleImputer(strategy="most_frequent")),
            ("encoder", OneHotEncoder(handle_unknown="ignore", sparse=False)),
        ]
    )

    preprocessor = ColumnTransformer(
        transformers=[
            ("num", numeric_transformer, NUMERIC_COLUMNS),
            ("cat", categorical_transformer, CATEGORICAL_COLUMNS),
        ],
        remainder="drop",
        sparse_threshold=0,
    )

    return Pipeline(steps=[("preprocessor", preprocessor), ("classifier", classifier)])


def train_and_select_best_model(X_train, X_test, y_train, y_test):
    models = {
        "Decision Tree": DecisionTreeClassifier(random_state=42),
        "Random Forest": RandomForestClassifier(random_state=42, n_estimators=100),
        "KNN": KNeighborsClassifier(n_neighbors=5),
        "XGBoost": xgb.XGBClassifier(use_label_encoder=False, eval_metric="logloss", random_state=42, n_estimators=
    }

    best_pipeline = None
    best_score = 0.0
    best_name = None

    for name, classifier in models.items():
        pipeline = build_pipeline(classifier)
        pipeline.fit(X_train, y_train)
        predictions = pipeline.predict(X_test)
        score = accuracy_score(y_test, predictions)
        print(f"{name} accuracy: {score * 100:.1f}%")

        if score > best_score:
            best_score = score
            best_pipeline = pipeline
            best_name = name

    if best_pipeline is None:
        raise RuntimeError("Failed to train any model.")

    print(f"Best model: {best_name} with accuracy {best_score * 100:.1f}%")
    return best_pipeline


def save_model(model):
    os.makedirs(MODEL_DIR, exist_ok=True)
    joblib.dump(model, MODEL_PATH)
    print(f"Saved best model to {MODEL_PATH}")


def main():
    df = load_data()
    X, y = preprocess_features(df)
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=42, stratify=y)
    best_model = train_and_select_best_model(X_train, X_test, y_train, y_test)
    save_model(best_model)
if __name__ == "__main__":
    main()
```

File: app.py

Full file contents:

```
import os
from flask import Flask, render_template, request
import joblib

app = Flask(__name__)
MODEL_PATH = os.path.join("models", "best_loan_model.pkl")

if not os.path.exists(MODEL_PATH):
    raise FileNotFoundError("Model file not found. Run 'python train_model.py' first.")

model = joblib.load(MODEL_PATH)
FEATURES = [
    "Gender",
    "Married",
    "Education",
    "Self_Employed",
    "ApplicantIncome",
    "CoapplicantIncome",
    "LoanAmount",
    "Loan_Amount_Term",
    "Credit_History",
    "Property_Area",
]


@app.route("/", methods=["GET"])
def index():
    return render_template("index.html")


@app.route("/predict", methods=["POST"])
def predict():
    form = request.form
    try:
        values = [
            form.get("gender", "Male"),
            form.get("married", "Yes"),
            form.get("education", "Graduate"),
            form.get("self_employed", "No"),
            float(form.get("applicant_income", "0") or 0),
            float(form.get("coapplicant_income", "0") or 0),
            float(form.get("loan_amount", "0") or 0),
            float(form.get("loan_amount_term", "360") or 0),
            int(form.get("credit_history", "1") or 0),
            form.get("property_area", "Urban"),
        ]
    except ValueError:
        return render_template(
            "result.html",
            decision="Invalid input",
            confidence="N/A",
            error_message="Please enter valid numeric values for income, loan amount, and loan term.",
        )

    input_data = [values]
    prediction = model.predict(input_data)[0]
    probability = None
    if hasattr(model, "predict_proba"):
        probability = model.predict_proba(input_data)[0].max()

    decision = "Loan Approved" if prediction == 1 else "Loan Rejected"
    confidence = f"{probability * 100:.1f}%" if probability is not None else "N/A"
```

```
    return render_template(
        "result.html",
        decision=decision,
        confidence=confidence,
        input_data=dict(zip(FEATURES, values)),
        error_message=None,
    )


if __name__ == "__main__":
    app.run(debug=True)
```

File: .gitignore

Full file contents:

```
venv/
__pycache__/
*.pyc
models/
.DS_Store
```

File: data\loan_data.csv

Full file contents:

```
Gender,Married,Education,Self_Employed,ApplicantIncome,CoapplicantIncome,LoanAmount,Loan_Amount_Term,Credit_History
Male,Yes,Graduate,No,5849,0,128,360,1,Urban,Y
Male,Yes,Graduate,No,4583,1508,128,360,1,Rural,N
Male,Yes,Not Graduate,Yes,3000,0,66,360,1,Urban,Y
Female,No,Graduate,No,6000,0,141,360,1,Urban,Y
Male,No,Graduate,Yes,5417,4196,267,360,1,Urban,Y
Male,Yes,Not Graduate,No,2333,1516,95,360,1,Rural,N
Female,Yes,Graduate,No,3036,2504,158,360,1,Urban,Y
Male,No,Graduate,No,4006,1526,168,360,1,Urban,N
Male,No,Not Graduate,No,12841,10968,350,360,1,Urban,Y
Female,Yes,Not Graduate,No,3200,0,200,360,0,Semiurban,N
```

File: templates\index.html

Full file contents:

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Smart Lender - Loan Prediction</title>
    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/css/bootstrap.min.css" rel="stylesheet">
  </head>
  <body>
    <div class="container py-5">
      <div class="text-center mb-4">
        <h1>Smart Lender</h1>
        <p class="lead">Predict loan eligibility using machine learning.</p>
      </div>

      <div class="card shadow-sm">
        <div class="card-body">
          <form action="/predict" method="post">
            <div class="row g-3">
              <div class="col-md-6">
                <label class="form-label">Gender</label>
                <select class="form-select" name="gender">
                  <option>Male</option>
                  <option>Female</option>
                </select>
              </div>
              <div class="col-md-6">
                <label class="form-label">Married</label>
                <select class="form-select" name="married">
                  <option>Yes</option>
                  <option>No</option>
                </select>
              </div>
              <div class="col-md-6">
                <label class="form-label">Education</label>
                <select class="form-select" name="education">
                  <option>Graduate</option>
                  <option>Not Graduate</option>
                </select>
              </div>
              <div class="col-md-6">
                <label class="form-label">Self Employed</label>
                <select class="form-select" name="self_employed">
                  <option>No</option>
                  <option>Yes</option>
                </select>
              </div>
              <div class="col-md-6">
                <label class="form-label">Applicant Income</label>
                <input type="number" step="1" class="form-control" name="applicant_income" required>
              </div>
              <div class="col-md-6">
                <label class="form-label">Coapplicant Income</label>
                <input type="number" step="1" class="form-control" name="coapplicant_income" value="0">
              </div>
              <div class="col-md-6">
                <label class="form-label">Loan Amount</label>
                <input type="number" step="0.1" class="form-control" name="loan_amount" required>
              </div>
              <div class="col-md-6">
                <label class="form-label">Loan Term (days)</label>
                <input type="number" step="1" class="form-control" name="loan_amount_term" value="360" required>
```

```
            </div>
            <div class="col-md-6">
              <label class="form-label">Credit History</label>
              <select class="form-select" name="credit_history">
                <option value="1">Good</option>
                <option value="0">No History</option>
              </select>
            </div>
            <div class="col-md-6">
              <label class="form-label">Property Area</label>
              <select class="form-select" name="property_area">
                <option>Urban</option>
                <option>Semiurban</option>
                <option>Rural</option>
              </select>
            </div>
          </div>
          <div class="mt-4 text-center">
            <button type="submit" class="btn btn-primary btn-lg">Predict Eligibility</button>
          </div>
        </form>
      </div>
    </div>
  </div>
</body>
</html>
```

File: templates\result.html

Full file contents:

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Smart Lender - Prediction Result</title>
    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/css/bootstrap.min.css" rel="stylesheet">
  </head>
  <body>
    <div class="container py-5">
      <div class="text-center mb-4">
        <h1>Prediction Result</h1>
      </div>
      <div class="card shadow-sm">
        <div class="card-body">
          {% if error_message %}
          <div class="alert alert-danger">{{ error_message }}</div>
          {% endif %}
          <h2 class="mb-3">{{ decision }}</h2>
          <p><strong>Confidence:</strong> {{ confidence }}</p>
          <hr>
          <h5>Input Details</h5>
          <ul class="list-group mb-4">
            {% for key, value in input_data.items() %}
            <li class="list-group-item"><strong>{{ key }}:</strong> {{ value }}</li>
            {% endfor %}
          </ul>
          <a href="/" class="btn btn-secondary">Back</a>
        </div>
      </div>
    </div>
  </body>
</html>
```